

阅读文献笔记

姜佳岩 · 围绕在线推理与 KV Cache 的三份材料 · 2026-09-27

1. SGLang: Efficient Execution of Structured Language Model Programs

多轮对话或不同请求有时会共用一段提示词。论文指出，每次都重新做这段 Prefill，会重复计算 KV。RadixAttention 用基数树保存 token 前缀及对应的 KV，新请求先匹配已有的最长前缀，再计算后面的新内容。缓存中的旧内容可以淘汰，调度时也可以考虑缓存命中情况。

读完这部分，我弄清了之前容易混淆的一点：复用的是注意力计算留下的 KV

状态，并不是直接拿之前生成的文字作答。只有 token

前缀相同，对应的计算结果才能复用。论文也提到，这种方式可以与分页注意力配合使用。

本机服务日志里有 RadixCache 的初始化信息，不过我没有统计缓存命中率。实验中的提示词是重复 hello 合成的，这可能让不同请求变得过于相似。因此，即便延迟较低，我也不能直接把它归因于 RadixAttention。

原文：Zheng et al., NeurIPS 2024, <https://arxiv.org/html/2312.07104>

2. Efficient Memory Management for Large Language Model Serving with PagedAttention

生成过程中，KV cache 会不断增长。给每个请求预留一整块连续显存，容易留下碎片和闲置空间，也会影响并发请求数。PagedAttention 将 KV

分成固定大小的逻辑块，映射到物理显存块，按需分配，还可以共享块。在我看来，这篇论文主要讨论的是 KV 如何存放和读取。

Prefill 处理已知输入，Decode 一轮轮生成新 token，两步都需要 KV。vLLM

后来还加入了基于块哈希的自动前缀缓存，可见分页存储与前缀复用可以同时使用。

原文：Kwon et al., SOSP 2023, <https://arxiv.org/html/2309.06180>

vLLM 前缀缓存设计：https://docs.vllm.ai/en/latest/design/prefix_caching/

3. Mooncake FAST'25 Trace Release README

这份数据采用 JSONL 格式，每行包含 timestamp、input_length、output_length、hash_ids 等字段，没有原始提示词。其中，hash_ids 提供了可供比较的前缀块信息。README 还说明，traces/ 下更新的 FAST'25 数据与 [arxiv-trace/mooncake_trace.jsonl](#) 这个历史单文件有所区分。

我用的是题目举例的历史单文件，一共有 23,608 条记录。按 $\text{input_length} + \text{output_length} \leq 3800$ 等长度条件筛选后，剩下 6,424 条，再用随机种子 42 从中抽取 20 条。合成提示词后，我用 Qwen 分词器核对实际长度。请求的到达间隔按指数分布生成，平均到达率为 0.5 次/秒。

这次合成 prompt 时只用了长度信息，没有根据 hash_ids 还原原有的前缀关系。output_length 被设为 max_tokens，但实际输出可能达不到这个数。脚本只记录了端到端 latency，没有测 TTFT。如果要继续比较缓存收益，还需要使用能体现共享前缀的输入，并设置关闭缓存等对照条件。

数据说明：<https://github.com/kvcache-ai/Mooncake/blob/main/FAST25-release/README.md>